

Travail de fin de session

Évaluation des biais linguistiques et des raccourcis statistiques dans un Transformer dédié à l'analyse de sentiment

Yves Courteau

SDD1004 - Apprentissage Automatique et applications

présenté à

Amir Saki

UQTR

20 avril 2025

Table des matières

Introduction	3
Contexte	3
Enjeu étudié	3
Présentation des fichiers du projet	4
Aperçus de l'architecture des transformers	5
Le coeur du transformer: le self-attention	5
Du texte brut à la prédiction	6
Démarche technique	7
Architecture du modèle	7
Préparation des données	8
Protocole d'entraînement	9
Évolution de l'entraînement	10
Définition des métriques d'évaluation	11
Résultat des métriques d'évaluation	12
Analyse de la confiance du modèle	13
Corrélations superficielles	15
Méthode d'exploration des erreurs	15
Analyse des mots les plus influents selon <i>LIME</i>	15
Analyse d'erreurs liées aux corrélations superficielles	16
Exemples adversariaux	17
Génération des exemples adversariaux	17
Effet des mots injectés sur les prédictions	17
Exemples de renversement de prédiction significatif	18
Analyse finale et proposition d'améliorations	21
Analyse des causes probables	21
Première cause	21
Deuxième cause	21
Troisième cause	22
Pour résumer	22
Proposition d'améliorations	22
Première amélioration	22
Deuxième amélioration	23
Troisième amélioration	23
Dernièrement	23
Conclusion	24
Références	25

Introduction

Contexte

Les *transformers* sont devenus des modèles très utilisés en traitement automatique du langage naturel, parce qu'ils permettent d'obtenir d'excellents résultats dans plusieurs types de tâches, comme dans le cas de mon présent travail avec l'analyse de sentiments. Le principal avantage est leur force d'analyse de relations entre les mots en prenant en considération le contexte des phrases dans leurs ensembles.

Au début, dans ce projet, j'avais commencé à développer moi-même en *Python* une version très simplifiée d'un *transformer*, surtout pour mieux comprendre la logique interne de ce type d'architecture. Par après, à la suite d'une discussion en classe, j'ai réalisé que le projet ne demandait pas une implémentation complète *from scratch*, mais que l'on devait utiliser un modèle déjà existant pour simplement le *fine-tune*. Après quelques recherches sur *Google*, j'ai donc choisi d'utiliser *DistilBERT* avec les données de *IMDB* qui semblaient être un *dataset* plutôt correct pour une première expérience avec les *transformers*. Cette réorientation a rendu le projet beaucoup plus réaliste dans le temps alloué, mais m'a aussi donné le goût de continuer plus tard, par intérêt personnel, à explorer plus en profondeur l'implémentation d'un *transformer from scratch* autant la partie *encoder* que la partie *decoder*.

Enjeu étudié

Les *transformers* aussi sont sujets aux problèmes de corrélations superficielles (*spurious correlation*). Au lieu d'obtenir une compréhension sémantique profonde et logique de la langue, le modèle peut devenir un simple *keyword finder* statistique. Il pourrait prendre par exemple le nom d'un acteur X et toujours l'associer à des critiques négatives, juste parce que l'acteur X est très présent dans des critiques négatives.

Cette dépendance à des indices statistiques non causal rend certain modèle vulnérable à des exemples adversariaux, autrement dit, des phrases simples à interpréter pour un humain, mais conçues pour mettre en erreur différents modèles en exploitant leurs biais.

Présentation des fichiers du projet

Le projet est organisé autour des fichiers et dossiers suivants :

- `README.md` : fichier de documentation du projet. Il explique l'installation de l'environnement, les dépendances qu'il faut installer et donne un aperçu de la structure du repo.
- `requirements.txt` : liste des dépendances Python pour l'exécution du fichier python et des IPYNB's du projet, contient entre autre `torch`, `datasets`, `transformers`, `scikit-learn`, `matplotlib` et `lime`.
- `train.py` : script principal pour l'entraînement du modèle. Il contient la tokenisation, le fine-tuning, le calcul des métriques et la sauvegarde des logs de l'entraînement.
- `evaluation_performance_du_modele.ipynb` : notebook consacré à l'évaluation des performances du modèle entraîné. Il contient la matrice de confusion, les métriques de classification et l'analyse de la confiance des différentes prédictions.
- `analyse.ipynb` : notebook d'analyse des corrélations superficielles et des exemples adversariaux. Fichier au coeur du projet, il sert à explorer les mots influents avec LIME et à étudier la sensibilité du modèle à certains mots injectés.
- `SDD1004_-_Yves_Courteau_-_Rapport.pdf` : version PDF du rapport final du projet que vous êtes en train de lire en ce moment même.
- `out/` : dossier de sortie contenant les résultats produits pendant l'entraînement et l'évaluation.
 - `training_metrics.json` : fichier JSON qui enregistre l'évolution des métriques d'entraînement et d'évaluation, notamment la *train loss*, la *eval loss* et l'*eval accuracy* (utilisé dans le notebook de l'évaluation de performanc du modèle).
 - `model_a/` : sous-dossier contenant les différents fichiers du modèle que j'ai retenu pour les étapes d'évaluation et d'analyse.

Aperçus de l'architecture des transformers

Le coeur du transformer: le self-attention

L'architecture d'un transformer repose en grande partie sur un mécanisme de self attention. En gros, lorsqu'un mot est analysé, le modèle ne le traite pas de façon isolée, mais regarde aussi quels autres mots de la phrase sont importants pour bien l'interpréter. Par exemple, le sens d'un mot peut changer selon tous les mots qui peuvent l'entourer, le self attention permet de le "comprendre".

Contrairement à des approches anciennes qui consistaient à lire le texte de façon séquentielle, les transformers peuvent considérer plusieurs relations dans une phrase en parallèle, ça permet de les rendre plus efficaces pour comprendre le contexte.

Le mécanisme de self-attention repose sur trois matrices dérivées de l'entrée, soit les requêtes (Q), les clés (K) et les valeurs (V), et se calcule comme suit :

$$\text{Self-Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

où les éléments sont définis comme suit :

Q	: matrice des requêtes (représente le mot en cours d'analyse)
K	: matrice des clés (représente tous les mots de la phrase)
V	: matrice des valeurs (contient l'information à extraire de chaque mot)
$QK^{\top}$	: produit scalaire entre requêtes et clés, mesure la similarité entre les mots
d_k	: dimension des vecteurs clés, utilisé pour stabiliser le gradient
$\sqrt{d_k}$	: facteur de normalisation pour éviter des valeurs trop grandes avant le softmax
softmax	: $\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_j \exp(s_{ij})}$ (conversion des scores en probabilités d'attention)

Du texte brut à la prédiction

Entrée et préparation du texte

- **Texte brut** : les données d'entrée sont des textes d'avis de films provenant du jeu de données IMDB, accessible depuis Hugging Face.
- **Tokenization** : le texte est découpé en parties plus petites appelées des tokens. Un token peut être un mot complet, un morceau de mot (subword) ou un signe de ponctuation.
- **Token IDs** : chaque token est associé à un identifiant numérique unique que le modèle va utiliser comme référence.
- **Padding dynamique** : les séquences mit en tokens ont pas toutes la même longueur. Un `DataCollatorWithPadding` est donc utilisé pour ajouter automatiquement du padding à l'intérieur de chaque batch.
- **Embeddings** : les identifiants de tokens sont transformés en vecteurs numériques denses qui représentent certaines propriétés sémantiques des mots.
- **Position embeddings** : l'information de la position est ajoutée aux embeddings pour que le modèle puisse tenir compte de l'ordre des différents tokens dans la séquence.

Traitement dans les blocs Transformer

- **Multi-Head Self-Attention** : le modèle compare chaque token avec les autres tokens de la séquence pour tenter de mieux comprendre le contexte. Dans ce projet, DistilBERT utilise 12 têtes d'attention par bloc de transformer.
- **Add & Layer Normalization** : la sortie de la self-attention est additionnée à l'entrée du bloc, puis normalisée afin de stabiliser l'apprentissage.
- **Feed-Forward Network (FFN)**: après l'attention, chaque représentation passe dans un petit réseau de neurones qui raffine l'information (pour DistilBERT : Linear, GeLU, Linear).
- **Add & Layer Normalization** : la sortie du réseau feed-forward est à nouveau additionnée à l'entrée précédente et après elle est normalisée.
- **Dropout** : DistilBERT applique ici un dropout de 0.1 afin de réduire le risque d'un surapprentissage des données IMDB.

- **Blocs Transformer répétés** : DistilBERT utilise 6 blocs transformer. Puisque chaque bloc contient 12 têtes d'attention, ça nous donne un total de 72 têtes d'attention dans l'ensemble du modèle DistilBERT.

Sortie

- **Classification head** : pour la tâche de classification binaire du projet (positif ou négatif), la représentation finale du premier token est utilisée pour produire les logits associés aux deux classes. Avec DistilBERT, la tête de classification applique un dropout de 0.2.
- **Logits** : il s'agit des scores bruts produits par la couche finale de classification avant leur conversion en probabilités par softmax.

Apprentissage du modèle

- **Loss** : on compare les logits aux vrais labels avec la fonction de perte CrossEntropyLoss (class=2), qui quantifie l'erreur de prédiction du modèle.
- **Optimisation** : après le calcul de la loss, les poids du modèle sont ajustés par backpropagation à l'aide de l'optimiseur AdamW, qui intègre aussi une régularisation avec weight decay pour limiter que les différents poids deviennent trop élevés.

Démarche technique

Architecture du modèle

J'avais commencé par lire sur les transformers et commencer l'implémentation d'un transformer minimaliste (enfin seulement la partie encoder). Après une discussion en classe, j'ai découvert que j'avais très mal compris le projet et que j'avais simplement à utiliser une librairie avec un modèle déjà implémenté comme celui de HuggingFace. Le modèle que j'ai utilisé est donc DistilBERT (distilbert-base-uncased), disponible via la bibliothèque de HuggingFace avec une tête de classification binaire (num_labels=2). DistilBERT est une version plus légère de BERT, donc un modèle de transformer de type encoder.

Ce qui veut dire qu'il peut comprendre et représenter le texte en contexte, il ne fait pas de la génération de texte, cette partie c'est la portion decoder du transformer qui s'en occupe, l'encoder et le decoder peuvent fonctionner ensemble, ou fonctionner séparément... Le fonctionnement de l'encoder repose en grande partie sur le mécanisme de self-attention, qui permet de tenir compte des relations entre les mots d'une phrase, même quand les mots sont éloignés l'un de l'autre. Dans le cadre de ce projet final du cours SDD1004, le modèle utilisé est déjà pré-entraîné et je le fine-tune sur les données de IMDB pour faire de l'analyse de sentiment.

Préparation des données

Les données du projet sont les données de IMDB disponible sur HuggingFace, ce sont des avis de films avec des étiquettes pour dire si l'avis est positif ou négatif, les données étaient déjà propre et ne demandait pas de faire de nettoyage. La première étape était d'appliquer le tokenizer relié à DistilBERT pour transformer toutes les critiques en une séquence de tokens qui fonctionne avec le modèle. Les textes sont découpés avec une longueur maximale de 256 tokens pour limiter le coût de mémoire, avoir un ordinateur portable plus puissant j'aurai pu utiliser du 512 par exemple.

Les données originales sont ensuite séparées en deux sous-ensembles, respectivement, un de 90% pour l'entraînement et un de 10% pour l'évaluation. J'ai utilisé un seed de 42 (j'aime bien la référence à ce nombre et le fait que beaucoup de scientifiques de données l'utilisent aussi) pour m'assurer de la reproductibilité.

Pour finir, la colonne de texte originale est enlevée après l'étape de tokenization pour seulement garder les entrées réellement consommées par le modèle. Un padding dynamique est utilisé par lot avec DataCollatorWithPadding pour uniformiser la longueur des séquences dans une batch (ça ajoute simplement des espaces vides à la fin des textes plus courts pour que toutes les séquences d'une même batch aient la même longueur).

Protocole d'entraînement

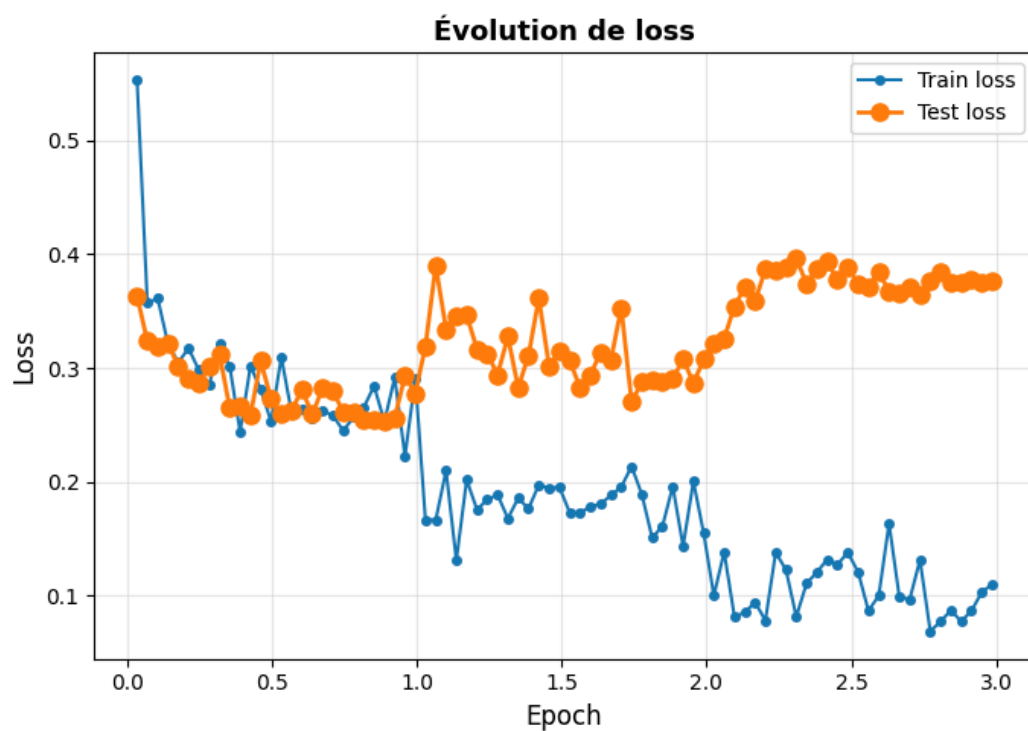
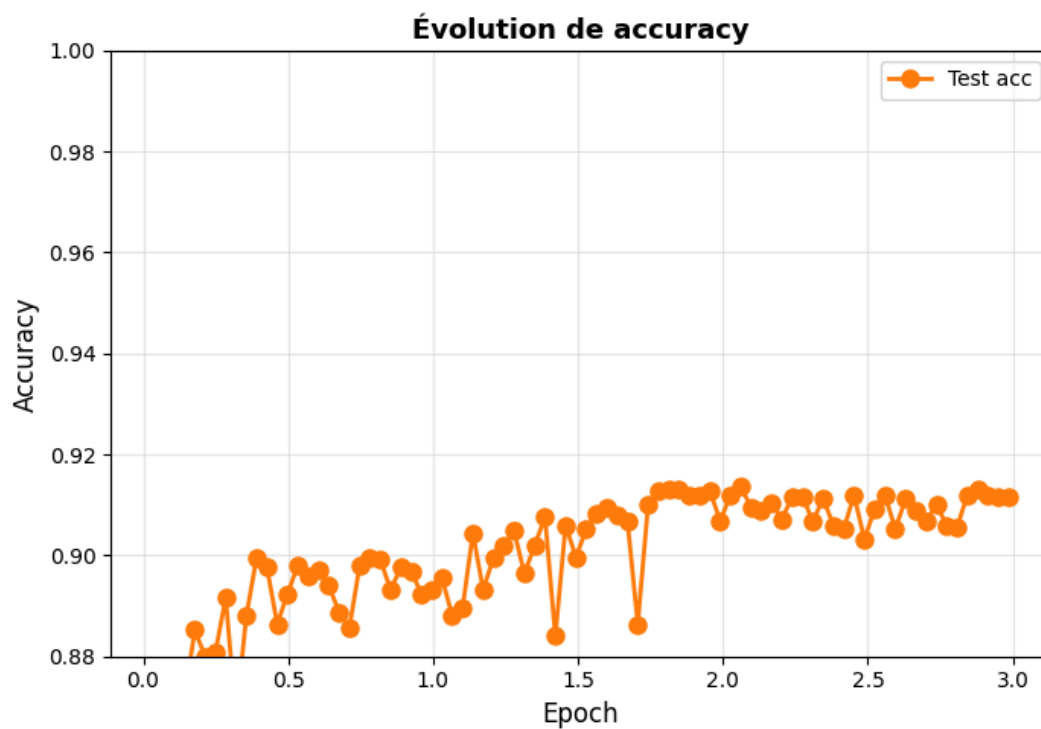
L'entraînement repose sur l'utilisation de l'API Trainer de Hugging Face, qui permet de faire automatiquement les étapes principales de fine-tuning, de l'évaluation et de la sauvegarde du modèle en fichiers...

Le fine-tuning est fait pendant 3 epochs (voit les données complètement 3 fois), avec un taux d'apprentissage de $2e-5$ (la taille des ajustements que le modèle fait à ses poids à chaque étape d'apprentissage) et un weight decay (régularisation qui empêche les poids de devenir trop lourd) de 0.01 pour essayer de limiter le surapprentissage.

Encore une fois, avec mon ordinateur portable j'ai des contraintes de performance matériel, j'ai donc dû réduire la taille des batches (4) pour l'entraînement, mais l'hyper paramètre gradient accumulation steps permet de simuler un batch plus grand sans surcharger la mémoire de l'ordinateur...

Après chaque 50 steps, une évaluation est faite utilisant une fonction que j'ai écrite pour afficher plusieurs métriques utiles, soit accuracy, precision, recall et F1. Cette partie m'a été très utile puisque avant de choisir mon modèle final (le checkpoint de l'epoch 2, que j'ai renommé `model_a`), j'avais fine-tuned une quinzaine de modèles différents avec des hyper paramètres différents, certains avec pratiquement dixaine d'epochs et je pouvais voir à chaque epoch si le loss devenait vraiment très grand et que l'accuracy n'augmentait pas assez pour accepter cette augmentation du loss entre autre.

Évolution de l'entraînement



Définition des métriques d'évaluation

Les métriques utilisées pour évaluer le modèle sont définies comme suit :

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1-score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$Cross-Entropy Loss = -\frac{1}{N} \sum_{i=1}^N \log(\text{softmax}(z_i)_{y_i})$$

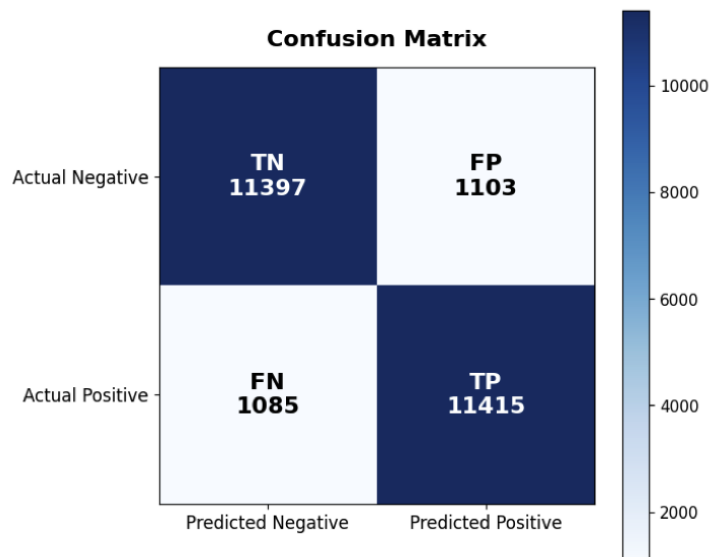
$$Confiance (Softmax) = \max_j \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

où les éléments sont définis comme suit :

- TP : vrais positifs — avis positifs correctement classés positifs
- TN : vrais négatifs — avis négatifs correctement classés négatifs
- FP : faux positifs — avis négatifs incorrectement classés positifs
- FN : faux négatifs — avis positifs incorrectement classés négatifs
- N : nombre total d'exemples dans l'ensemble d'évaluation
- y_i : étiquette réelle de l'exemple i (0 = négatif, 1 = positif)
- z_i : vecteur des logits bruts produits par le modèle pour l'exemple i
- z_j : logit brut produit pour la classe j
- K : nombre de classes (ici $K = 2$, soit positif et négatif)
- $\max_j$: valeur maximale sur les classes — correspond à la confiance retenue

Résultat des métriques d'évaluation

Voici un récapitulatif des principales métriques obtenues dans le fichier *evaluation_performance_du_modele.ipynb*, en commençant avec la matrice de confusion:



- *Loss* : 0.2717
- *Accuracy* : 0.9125
- *Precision* : 0.9119
- *Recall* : 0.9132
- *F1-score* : 0.9125

En moyenne, le modèle donne une probabilité de 96.93% à la classe qu'il va choisir, que ce soit positif ou négatif.

- *Confiance moyenne* : 0.9663
- *Confiance minimale* : 0.5003
- *Confiance maximale* : 0.9979

Analyse de la confiance du modèle

Les 5 predictions les plus confiante:

Confiance: 0.997878
Vrai label: 1 | **Prediction:** 1
Texte: "Boogie Nights" is a masterpiece it tells a gr...

Confiance: 0.997875
Vrai label: 1 | **Prediction:** 1
Texte: I have not seen a better comedy than the King ...

Confiance: 0.997874
Vrai label: 1 | **Prediction:** 1
Texte: This was the best movie ever has seen on "Germ...

Confiance: 0.997847
Vrai label: 1 | **Prediction:** 1
Texte: This show is my absolute favorite. This show i...

Confiance: 0.997822
Vrai label: 1 | **Prediction:** 1
Texte: This is an excellent movie and should be prese...

Les 5 predictions les moins confiante:

Confiance: 0.500274
Vrai label: 1 | **Prediction:** 1
Texte: One of the best 'guy' movies I've ever seen ha...

Confiance: 0.500298
Vrai label: 1 | **Prediction:** 0
Texte: Until today, I thought there only three people...

Confiance: 0.500324
Vrai label: 1 | **Prediction:** 0
Texte: I just got the DVD for Hardware Wars, in a shi...

Confiance: 0.500454
Vrai label: 1 | **Prediction:** 0
Texte: Bud Abbott and Lou Costello always had a good ...

Confiance: 0.500504
Vrai label: 1 | **Prediction:** 1
Texte: The last of the sequels,not counting Abbott an...

5 erreurs avec la plus forte confiance:

Confiance: 0.9975

Vrai label: 0 | **Prediction:** 1

Texte: This is definitely one of the best Kung fu movies in the history of Cinema. The screenplay is really well done (which is not often the case for this type of movies) and you can see that Chuck (in one of his first role) is a great actor. The final fight with the sheriff deputy in the bullring is a masterpiece!

Confiance: 0.9974

Vrai label: 0 | **Prediction:** 1

Texte: 'Ernest Saves Christmas' is comedian Ernest's Christmas special film. In this film, Ernest has to find a successor to Santa Claus in order for Christmas to continue. Along the way, he meets a young girl who is a thief and who ends up stealing something very important of Santa's (I won't tell you what that is; you'll just have to watch the film for yourself). Like most Ernest films, there is a lot of slapstick humor involved.
This film is probably one of Ernest's best films. It's funny

Confiance: 0.9973

Vrai label: 1 | **Prediction:** 0

Texte: Bears about as much resemblance to Dean Koontz's novel as Jessica Simpson does to a rocket scientist. If you've read the book, I suggest you put it as far out of your mind as possible before watching the movie.
Watchers is your typical "Boy meets dog, dog turns out to be super-intelligent government lab experiment, dog and boy are pursued by super-intelligent and emotionally disturbed monster created by same lab, and, oh yeah, did I mention the shady government agents pursuing the mon

Confiance: 0.9972

Vrai label: 0 | **Prediction:** 1

Texte: This movie was pure genius. John Waters is brilliant. It is hilarious and I am not sick of it even after seeing it about 20 times since I bought it a few months ago. The acting is great, although Ricki Lake could have been better. And Johnny Depp is magnificent. He is such a beautiful man and a very talented actor. And seeing most of Johnny's movies, this is probably my favorite. I give it 9.5/10. Rent it today!

Confiance: 0.9972

Vrai label: 0 | **Prediction:** 1

Texte: This has to be one of the all time greatest horror movies. Charles Band made the best movie of 96' in this little seen gem. Highly realistic and , incredibly stylised- with a visual flair David Fincher would envy, its not hard to see why Band went on to make such classics as 'Killjoy 2: Deliverance From Evil', 'The Regina Pierce Affair', 'Virgins of Sherwood Forest', and 'Timegate: Tales of the Saddle Tramps'.
With a highly sophisticated story- a tiny body with a large head controls

Corrélations superficielles

Méthode d'exploration des erreurs

Pour identifier les erreurs qui pourraient être liées à des corrélations superficielles, j'ai pris une approche exploratoire basée sur l'utilisation de *LIME*, une librairie *Python* qui sert à interpréter localement des prédictions d'un modèle, en identifiant les mots qui influencent le plus la décision pour un texte donné.

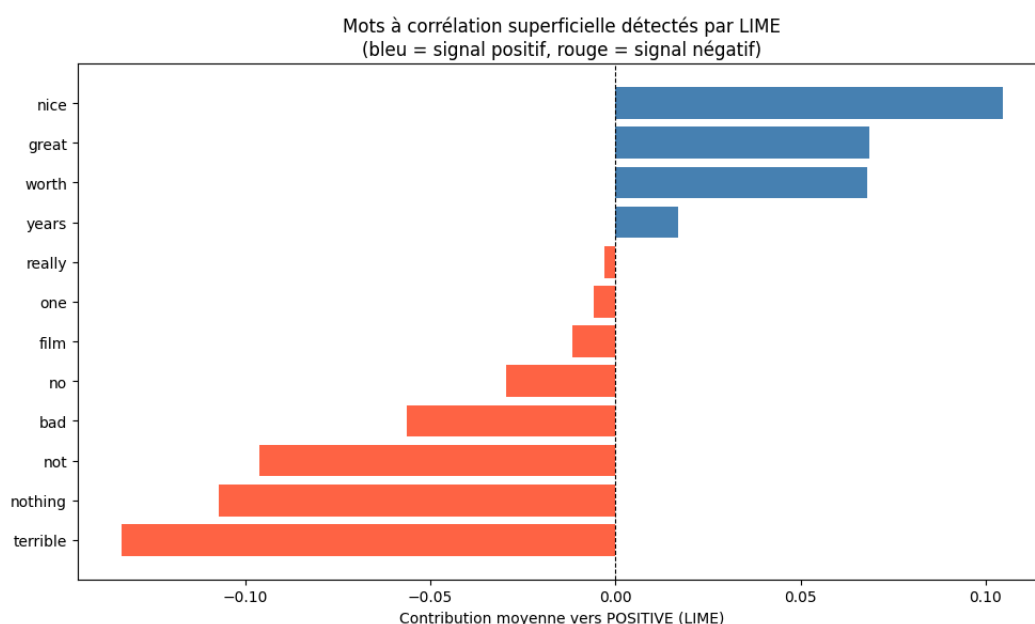
L'idée consiste à observer, sur un ensemble d'avis des données *IMDB*, quels mots ont l'air d'avoir le plus influencé localement la décision du modèle.

Analyse des mots les plus influents selon *LIME*

J'ai utilisé un échantillon équilibré de 50 avis provenant des données *IMDB*, soit 25 avis positifs et 25 avis négatifs, choisis aléatoirement mais avec un seed fixe pour que ce soit reproductible à chaque exécution. Pour chaque texte de l'échantillon, *LIME* estime les mots qui contribuaient le plus à pousser la prédiction obtenue par le modèle. Après j'ai regroupé les contributions par mot pour voir lesquelles revenaient souvent avec une influence positive ou négative.

L'agrégation fait ressortir certains mots qui jouent souvent le rôle de décision à savoir si le modèle va prédire que l'avis est positif ou négatif. Les mots "not", "nothing", "no", "film", "bad", "really", "terrible" et "one" reviennent comme des signaux plutôt négatifs, pendant que "nice", "great", "worth", et "years" reviennent comment des signaux plutôt positifs.

La visualisation suivante confirme que certains mots semblent influencer la décision du modèle de manière récurrente, ce qui veut dire qu'ils peuvent agir comme des mots ayant une forte influence sur la prédiction.



Analyse d'erreurs liées aux corrélations superficielles

Les erreurs de classification observées sur ce même échantillon ont été croisées avec les mots suspects trouvés à l'étape précédente. J'ai pu isoler deux erreurs potentielles sur l'échantillon des 50 avis.

Dans le premier cas, un avis réellement positif a été classé comme négatif, et le mot "not" apparaît comme un élément influent sur la décision prise par le modèle.

Dans le deuxième cas, un avis réellement négatif a été classé comme positif, avec le mot "nice" qui a joué un rôle influent dans la prédiction obtenu par le modèle.

Les deux exemples ne suffisent pas, à eux seuls pour démontrer que le modèle repose seulement sur des corrélations superficielles. Mais ça montre quand même que dans certains cas, la présence de certains mots semblent peser davantage dans la prédiction que l'interprétation complète de l'avis. Les résultats observés laissent penser qu'une partie des décisions du modèle repose sur des mots plutôt simples, et je comprends mieux l'utilité de la prochaine étape demandée pour ce travail, soit l'analyse d'exemples adversariaux.

Exemples adversariaux

Génération des exemples adversariaux

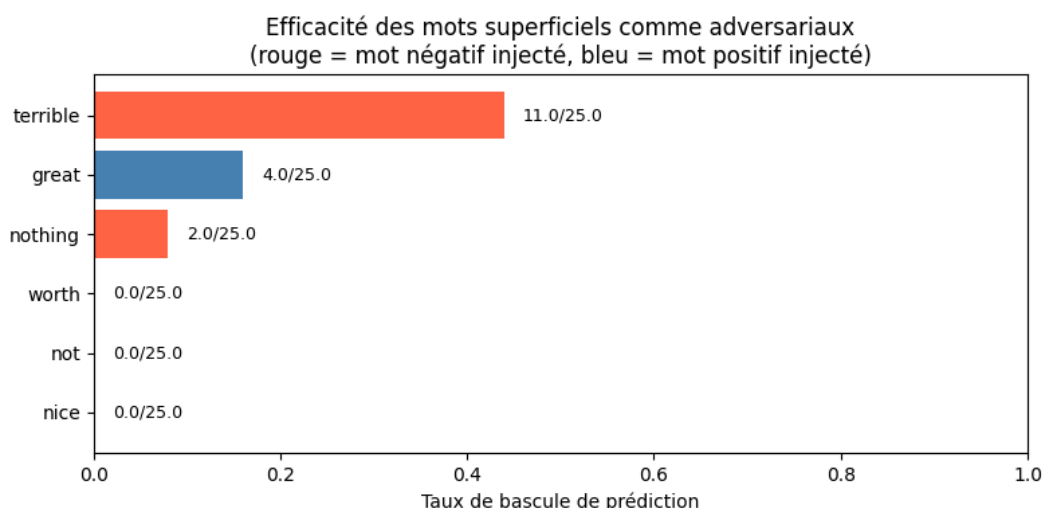
Pour faire les exemples adversariaux, j'ai premièrement choisi des avis de base déjà bien classifiés par le modèle fine-tuned. Plus précisément, j'ai gardé 25 avis positifs et 25 avis négatifs des données IMDB, en gardant seulement ceux dont la prédiction était correcte et assez confiante. Le bût était de faire partie de textes sur lesquels le modèle semblait déjà relativement stable pour que tout changement observé par après soit plus facile à interpréter.

Avec les avis de base, j'ai injecté un seul mot dans le texte. J'ai choisis les mots qui semblait avoir le plus d'influence sur les prédictions du modèle, je me suis simplement fier au résultat de mon analyse que j'ai réaliser sur les corrélations superficielles avec l'aide de LIME dans la section précédente, soit "terrible", "nothing", "not" pour les négatifs et "nice", "great", "worth" pour les positifs. L'idée était de vérifier si l'ajout d'un mot isolé, sans réécrire complètement l'avis pouvait suffire à modifier la prédiction du modèle.

Effet des mots injectés sur les prédictions

Au total, 150 exemples adversariaux ont été générés, parce que chaque avis de base a été testé avec trois mots injectés, si l'avis est positif, on injecte les mots négatif et vice versa. Pour les 150 cas, 17 ont provoqué une bascule complète de la prédiction du modèle. Aussi, le delta moyen absolu de 0.1169, ce qui montre que même quand la classe prédite ne change pas complètement, l'ajout d'un seul mot peut quand même influencer drastiquement la confiance du modèle sur une prédiction.

Le schéma suivant montre que tous les mots injectés n'ont pas eu le même effet. Le mot "terrible" est celui qui provoque le plus de bascules, avec un taux de réussite beaucoup plus élevé que les autres mots utilisés pour les différents tests. Le mot "great", lui aussi, semble avoir un certain impact sur les prédictions du modèle.



Exemples de renversement de prédiction significatif

Plusieurs cas observés montrent bien le phénomène. Par exemple, certains avis au début étaient classés positif avec une confiance très significative, même quelques fois au-dessus de 0.98, ils deviennent négatifs après l'injection du mot "terrible" dans le texte. Dans certains cas, la probabilité de la classe positive tombe de manière drastique, par exemple de 0.9829 à 0.0169. Ça montre qu'un seul mot avec une forte connotation peut des fois suffire pour renverser complètement une décision faite par le modèle.

Le phénomène existe aussi dans l'autre direction, certain avis qui au début était classé négatif par le modèle change pour être classé faussement en positif après l'ajout du mot "great". Par exemple, un avis négatif avec une confiance positive au départ a 0.1157 devient positif avec une confiance de 0.8320 après l'injection du mot. Dans l'ensemble, les résultats montrent que le modèle peut être très sensible à des mots isolés, ce qui appuie l'idée qu'une partie des décisions du modèle est basé des fois sur des indicateurs superficiels à la place d'une compréhension complète du texte.

Résultat des basculements de prédictions après injection d'un mot

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9829)
Après: NEGATIVE (conf POS=0.0169)
Delta: -0.9660
Extrait: This is my first Deepa Mehta film....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9939)
Apres: NEGATIVE (conf POS=0.2977)
Delta: -0.6962
Extrait: I finally watched the third film in Mehta's trilogy: "Fire"....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9968)
Apres: NEGATIVE (conf POS=0.3785)
Delta: -0.6183
Extrait: one of best movies ever....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9939)
Apres: NEGATIVE (conf POS=0.1753)
Delta: -0.8186
Extrait: It is more a subtle story of the fact that in Indian household how most decisions are taken by the man, how no attention...

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9842)
Apres: NEGATIVE (conf POS=0.2134)
Delta: -0.7708
Extrait: Its a very sensitive portrayal of life with unquenched or constrained desires....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9836)
Apres: NEGATIVE (conf POS=0.1241)
Delta: -0.8595
Extrait: This was a bold movie to hit Indian cinemas when it was released....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9847)
Apres: NEGATIVE (conf POS=0.1664)
Delta: -0.8183
Extrait: 'Water' (2005), the final part of Toronto-based Indian film-director Deepa Mehta's elemental trilogy has been finally co...

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9904)
Apres: NEGATIVE (conf POS=0.015)
Delta: -0.9754
Extrait: For all that has been said about the subject matter, and the controversy that surrounded it, please do not overlook what...

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9255)
Apres: NEGATIVE (conf POS=0.0129)

Delta: -0.9126
Extrait: The movie is really about choices....

Type: POS + injection NEG
Mot: 'nothing'
Avant: POSITIVE (conf POS=0.9255)
Apres: NEGATIVE (conf POS=0.3468)
Delta: -0.5787
Extrait: The movie is really about choices....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.9661)
Apres: NEGATIVE (conf POS=0.0387)
Delta: -0.9274
Extrait:

Her Excellency Madam Shabana Azmi has worked in countless movies over life time....

Type: POS + injection NEG
Mot: 'nothing'
Avant: POSITIVE (conf POS=0.9661)
Apres: NEGATIVE (conf POS=0.1535)
Delta: -0.8126
Extrait:

Her Excellency Madam Shabana Azmi has worked in countless movies over life time....

Type: POS + injection NEG
Mot: 'terrible'
Avant: POSITIVE (conf POS=0.988)
Apres: NEGATIVE (conf POS=0.4989)
Delta: -0.4891
Extrait: This is a very, very odd film....

Type: NEG + injection POS
Mot: 'great'
Avant: NEGATIVE (conf POS=0.0403)
Apres: POSITIVE (conf POS=0.5897)
Delta: +0.5494
Extrait: Worth the entertainment value of a rental, especially if you like action movies....

Type: NEG + injection POS
Mot: 'great'
Avant: NEGATIVE (conf POS=0.026)
Apres: POSITIVE (conf POS=0.6522)
Delta: +0.6262
Extrait: This cheap, grainy-filmed Italian flick is about a couple of inheritors of a manor in the Italian countryside who head u...

Type: NEG + injection POS
Mot: 'great'
Avant: NEGATIVE (conf POS=0.0528)
Apres: POSITIVE (conf POS=0.6996)
Delta: +0.6468
Extrait: Every movie I have PPV'd because Leonard Maltin praised it to the skies has blown chunks! Every single one! When will I ...

Analyse finale et proposition d'améliorations

Analyse des causes probables

Les résultats obtenus au travers des deux étapes précédentes, soit l'analyse LIME et la génération d'exemples adversariaux, montrent une cause principale. Le modèle a appris à associer certains mots à une classe de façon très directe, sans prendre en considération le contexte global de la phrase...

Première cause

La première cause est un biais lexical d'entraînement. Dans les données d'IMDB, certains mots reviennent de façon très déséquilibrée selon la classe. Un mot comme "terrible" se retrouve très fréquemment dans les avis négatifs et "great" dans les avis positifs. Après le fine-tuning, le modèle a appris les associations statistiques au point qu'un seul mot peut être suffisant à renverser une prédiction, même si le reste de la phrase va dans le sens totalement opposée. C'est exactement ce que montrent les résultats des exemples adversariaux, sur 150 exemples générés, 17 bascules complètes ont été provoquées juste par l'ajout d'un mot seul en début de texte, sans rien modifier d'autre.

Deuxième cause

La deuxième cause est liée à la nature même du mécanisme d'attention des transformers. Le self attention est en théorie capable de prendre en compte le contexte complet d'une phrase, mais il semble être quand même sensible aux mots sémantiquement forts. Un mot comme "terrible" capte beaucoup d'attention et peut même dominer la représentation finale même quand il est en contradiction totale avec le reste du texte de l'avis. Les cas observés dans mon analyse prouvent qu'un avis positif avec une confiance de départ très élevée peut chuter drastiquement après l'ajout d'un seul mot en tête du texte. Le modèle semble parfois accorder vraiment trop d'importance à ce nouveau mot injecter par rapport au sens global de la phrase.

Troisième cause

La troisième cause, elle est plus subtile et n'est pas réellement démontrée dans mon IPYNB due à des limitation matériel (mon ordinateur portable), c'est la longueur maximale des séquences utilisées. J'ai dû entraîner le modèle avec une limite de 256 tokens, donc pour les avis plus longs, la fin du texte est tronquée. Un mot injecté au début n'est pas toujours présent dans la fenêtre d'attention, ce qui augmente artificiellement son influence sur la prédiction finale....

Pour résumer

Finalement, les deux erreurs identifiées dans l'échantillon de 50 avis démontrent bien les causes. Dans le premier cas, le mot “not” dans un avis globalement positif a suffi pour faire changer la prédiction vers négatif pendant que l'avis parlait positivement du film de John Carpenter. Dans le second cas, le mot “nice” dans une review clairement négative à donner une prédiction positive... Dans les deux cas, le modèle semble accorder trop d'importance à un mot isolé plutôt qu'au sens global de l'avis.

Proposition d'améliorations

Plusieurs choses peuvent être envisagées pour rendre le modèle moins sensible aux corrélations superficielles.

Première amélioration

La première amélioration que je propose est sûrement celle qui aurait le plus d'impact. À la place de faire une analyse avec LIME sur seulement 50 avis, je prendrais littéralement l'entièreté des données IMDB et je trouverais tout les mots qui ont une très grande influence pour construire une liste et après j'utiliserais cette même liste de mot influent pour générer des exemple adversariaux que j'ajouterais au donnée IMDB afin d'augmenter le dataset utilisé à la phase d'entraînement.

Le dataset aurait donc des exemples similaires à ceux que j'ai produits dans

le IPYNB mais assigné au bon label, en gros, le dataset final aurait les 25000 lignes déjà présentes, mais en plus, il aurait plusieurs avis positifs contenant des mots négatifs et vice versa pour venir renforcer son entraînement. Ça forcerait le modèle à apprendre et de ne pas se fier uniquement à la présence de cette liste de mots isolés ultra influents. Je n'ai pas essayé mais je suis persuadé que mon intuition est correcte pour améliorer la robustesse du modèle.

Deuxième amélioration

La deuxième amélioration serait d'augmenter la longueur maximum des séquences pour le fine-tuning. Avec seulement 256 tokens, comme mentionné précédemment, les avis plus long sont tronqués, ce qui donne un avantage disproportionné aux mots qui apparaissent tôt dans le texte. Augmenter à 512 tokens, permettrait au modèle d'intégrer plus de contexte avant de prendre une décision.

Troisième amélioration

Une troisième amélioration pourrait être d'utiliser un modèle plus robuste ou plus riche que DistilBERT, comme BERT-base ou RoBERTa pour vérifier si une architecture plus puissante peut mieux comprendre le contexte global et être moins impactée par les mots isolés très influents. La comparaison avec un modèle plus complet pourrait aider à voir si une partie du problème vient des limites du modèle lui-même.

Dernièrement

Dernièrement, je pourrais augmenter davantage le dataset d'IMDB. Je pourrais manuellement écrire des avis et leur donner un label moi-même, des avis avec de la négation et des contrastes de sens plus complexes, enfin, simple qui reste simple et cohérent pour un humain mais qui pourrait avoir tendance à être complexe pour un modèle transformer.

Par exemple:

- *I expected something terrible, but it was actually really great.*
- *The movie looks nice at first, but it becomes boring and empty.*

Conclusion

Ce projet avait pour objectif d'explorer les limites du modèle transformer sur les données d'IMDB, plus précisément face aux corrélations superficielles et aux exemples adversariaux.

Le modèle DistilBERT une fois fine tuned obtient des métriques excellente, avec un accuracy de 91.25% et un F1-score de même valeur, ce qui confirme qu'il peut effectuer correctement la tâche d'analyse de sentiment dans la majorité des cas. Par contre, l'analyse faite avec LIME montre aussi une évidence, celle que certaines décisions du modèle se base plus sur la présence de mot isolés à forte connotation, comme "terrible" ou encore "nice", que sur une compréhension réelle du contexte du texte dans son ensemble.

La génération d'exemple adversariaux montre après une confirmation cette fragilité que le modèle peut avoir. En injectant un seul mot en début d'un texte d'avis sans même modifier quoi que ce soit d'autre, 17 renversements de prédiction ont été provoquées sur 150 tentatives soit dans 11.33% des cas, et le delta moyen de confiance, soit 0.1169, montre que même les cas sans renversement de prédiction complet sont affecté. Les résultats montrent clairement que la robustesse d'un modèle ne peut pas être évaluée juste avec ses métriques.

Ce projet m'a permis d'apprendre plusieurs choses, pour être totalement honnête au départ j'ai dû Google les termes corrélation superficiel et exemple adversarial. Le fait qu'au départ j'ai mal compris ce qui était demandé et que j'avais tenté de commencer à apprendre l'architecture derrière les transformers m'a donné envie d'en apprendre davantage sur le sujet une fois que le cours sera terminé. Je comprends aussi la raison de ce qui était demandé dans le travaux, parfois même sans formule mathématique on peut chercher des effets causaux comme c'est le cas avec ce petit mot isolé qui influent énormément sur la décision prise par un modèle plutôt complexe.

Références

- Hugging Face. *Hugging Face website*.
 - <https://huggingface.co>
- Hugging Face. *IMDb dataset*.
 - <https://huggingface.co/datasets/stanfordnlp/imdb>
- Hugging Face. *DistilBERT base uncased model card*.
 - <https://huggingface.co/distilbert/distilbert-base-uncased>
- Hugging Face. *Transformers documentation*.
 - <https://huggingface.co/docs/transformers/index>
- Hugging Face. *Hugging Face YouTube channel*.
 - <https://www.youtube.com/@HuggingFace>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*.
 - <https://arxiv.org/pdf/1706.03762>
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “*Why Should I Trust You?*”: *Explaining the Predictions of Any Classifier*.
 - <https://arxiv.org/pdf/1602.04938>
 - <https://github.com/marcotcr/lime>
- Notes de cours *SDD1004*.